Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №1 по курсу «Дискретный анализ»

Студент: Д. А. Широбоков
Преподаватель: С. А. Михайлова
Группа: М8О-301БВ
Дата:
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №7

Задача: Имеется натуральное число n. За один ход с ним можно произвести следующие действия:

- вычесть единицу;
- разделить на два;
- разделить на три.

При этом стоимость каждой операции — текущее значение n. Стоимость преобразования — суммарная стоимость всех операций в преобразовании. Вам необходимо с помощью последовательностей указанных операций преобразовать число n в единицу таким образом, чтобы стоимость преобразования была наименьшей. Делить можно только нацело.

Формат ввода

В первой строке задано $2 \leq n \leq 10^7$.

Формат вывода

Выведите на первой строке искомую наименьшую стоимость. Во второй строке должна содержаться последовательность операций. Если было произведено деление на 2 или на 3, выведите `/2` (или `/3`). Если же было вычитание, выведите `-1`. Все операции выводите, разделяя пробелом.

1 Описание

Требуется преобразовать натуральное число n в единицу с помощью операций «вычесть единицу», «разделить на два» и «разделить на три» (деление только нацело). Стоимость каждой операции равна текущему значению числа, а стоимость преобразования — сумме стоимостей всех операций. Нужно найти преобразование минимальной стоимости и вывести его стоимость и последовательность операций.

Задача решается методом динамического программирования. Жадный выбор здесь не подходит: например, для $n = 10$ деление на два даёт число 5, из которого выгоднее вычесть единицу, чем делить дальше, а локально лучший шаг не всегда ведёт к глобальному оптимуму. При этом выполняется свойство оптимальной подструктуры: если оптимальное преобразование числа i начинается с некоторой операции, то остаток пути является оптимальным преобразованием полученного числа.

Пусть $dp[i]$ — минимальная стоимость преобразования числа i в единицу. Тогда

$$dp[1] = 0, \qquad dp[i] = i + \min \begin{cases} dp[i-1], & \\ dp[i/2], & \text{если } i \bmod 2 = 0, \\ dp[i/3], & \text{если } i \bmod 3 = 0. \end{cases}$$

Слагаемое i появляется потому, что первая операция применяется к числу i и стоит ровно i. Значения вычисляются в порядке возрастания i, так как все $dp[j]$, от которых зависит $dp[i]$, имеют $j < i$ и уже посчитаны.

Для восстановления ответа вместе со стоимостью в каждой ячейке хранится операция, на которой достигается минимум. После вычисления таблицы начинаем с $i = n$ и пока $i > 1$ выводим сохранённую операцию и переходим к $i - 1$, $i/2$ или $i/3$ соответственно. Операции выводятся в порядке их применения, так что дополнительного разворота не требуется.

Сложность по времени — $O(n)$, так как каждое значение $dp[i]$ вычисляется за $O(1)$. Сложность по памяти — тоже $O(n)$: при $n \le 10^7$ таблица из элементов по 8 байт занимает около 80 МБ.

2 Исходный код

Операции описаны перечислением *operaion* с тремя значениями: *minus_1*, *div_2* и *div_3*. Для ячейки таблицы создана структура *elem*, где хранятся стоимость *price* и операция *op*, на которой достигнут минимум.

Таблица *dp* заполняется для i от 2 до n. Сначала берётся вариант «вычесть единицу», затем, если i делится на 2 или на 3 и соответствующая стоимость строго меньше текущей, ячейка обновляется вариантом с делением. В конце к стоимости прибавляется i. Базовые элементы $dp[0]$ и $dp[1]$ имеют стоимость 0; $dp[0]$ не используется и нужен лишь для того, чтобы индексы совпадали со значениями n.

```cpp
#include <iostream>
#include <vector>

enum operaion {
    minus_1,
    div_2,
    div_3
};

struct elem {
    int price = 0;
    operaion op;

    elem() {}
    elem(int p, operaion o) : price(p), op(o) {}
};

int main(void) {
    int n;
    std::cin >> n;

    std::vector<elem> dp {elem{}, elem{}};
    dp.reserve(n+1);

    for (int i = 2; i <= n; ++i) {
        dp.emplace_back(dp[i-1].price, minus_1);
        if (i % 2 == 0 && dp[i / 2].price < dp[i].price) {
            dp[i] = {dp[i / 2].price, div_2};
        }
        if (i % 3 == 0 && dp[i / 3].price < dp[i].price) {
            dp[i] = {dp[i / 3].price, div_3};
        }
        dp[i].price += i;
    }

    std::cout << dp[n].price << '\n';

```

```
38      for (int i = n; i > 1;) {
39          if (dp[i].op == minus_1) {
40              std::cout << "-1 ";
41              --i;
42          } else if (dp[i].op == div_2) {
43              std::cout << "/2 ";
44              i /= 2;
45          } else if (dp[i].op == div_3) {
46              std::cout << "/3 ";
47              i /= 3;
48          }
49      }
50  }
```

3 Консоль

```
laba-7>cmake -S . -B build
laba-7>cmake --build build
laba-7>..build/Debug/main.exe
10
21
/2 -1 /2 -1
```

Проверка для $n = 10$: операции $10 \to 5 \to 4 \to 2 \to 1$ стоят $10 + 5 + 4 + 2 = 21$.

4 Тест производительности

Тест производительности сравнивает решение на динамическом программировании с наивным рекурсивным перебором без запоминания подзадач. Наивный вариант считает стоимость по тому же соотношению, что и ДП: $f(n) = n + \min(f(n-1), f(n/2), f(n/3))$ с учётом делимости, но каждый раз вычисляет f заново. Поэтому одни и те же значения пересчитываются многократно, а количество рекурсивных вызовов $T(n) = T(n-1) + T(n/2) + T(n/3) + 1$ растёт сверхполиномиально.

Программа собрана компилятором MSVC 19.51 в конфигурации Release (флаг `/O2`), стандарт C++20. Время измерялось с помощью `std::chrono::steady_clock`, каждый замер повторялся 5 раз, в таблице указано среднее значение. Измерялось только вычисление минимальной стоимости, без чтения ввода и без вывода.

n	ДП, с	Наивная рекурсия, с	Отношение
50	0,000000760	0,000005600	7,4
75	0,000000520	0,000023280	44,8
100	0,000000500	0,000076760	153,5
500	0,000001580	0,254238580	160 910,5
750	0,000003880	3,056941520	787 871,5

n	ДП, с	Рост к предыдущему	Время на элемент, нс
10^3	0,000008860	—	8,86
10^4	0,000040460	4,57	4,05
10^5	0,000310240	7,67	3,10
10^6	0,004571280	14,74	4,57
10^7	0,048045160	10,51	4,80

```
laba-7>cmake -S . -B build
laba-7>cmake --build build --config Release
laba-7>build/Release/bench.exe
n       ДП,с            наивная,с   во сколько
50      0.000000760      0.000005600  7.4
75      0.000000520      0.000023280  44.8
100     0.000000500      0.000076760  153.5
500     0.000001580      0.254238580  160910.5
750     0.000003880      3.056941520  787871.5
n       ДП,с
1000    0.000008860
10000   0.000040460
```

```
100000  0.000310240
1000000 0.004571280
10000000 0.048045160
```

Из первой таблицы видно, что время наивной рекурсии растёт очень быстро. При увеличении n с 50 до 75 оно возрастает примерно в 4 раза, с 75 до 100 — в 3,3 раза, при росте n с 100 до 500 (в 5 раз) — более чем в 3000 раз, а при росте с 500 до 750 (в 1,5 раза) — ещё в 12 раз. При $n = 750$ перебор работает около трёх секунд, а при $n = 10^7$ он заведомо неприменим. Такой рост, намного более быстрый, чем у любого степенного закона, подтверждает, что без запоминания подзадач задача решается за сверхполиномиальное время. Отношение времени перебора ко времени ДП вырастает с 7,4 раза при $n = 50$ до почти $8 \cdot 10^5$ раз при $n = 750$, поэтому запоминание подзадач необходимо.

Во второй таблице время ДП измерено для n от 10^3 до 10^7. На малых n (до 100) время ДП составляет доли микросекунды и сравнимо с погрешностью таймера и накладными расходами на выделение памяти под вектор, поэтому небольшие колебания (например, время для $n = 75$ меньше, чем для $n = 50$) не отражают реальной зависимости. На больших n картина устойчива: при каждом увеличении n в 10 раз время растёт в 4–15 раз, а время, приходящееся на один элемент таблицы, остаётся в пределах 3–5 нс. Это согласуется с оценкой $O(n)$: каждая ячейка вычисляется за $O(1)$.

Рост времени в 14,7 раза при переходе от 10^5 к 10^6 немного превышает ожидаемые 10 раз. Вероятная причина в том, что при $n = 10^5$ таблица (около 800 КБ) ещё помещается в кэш процессора, а при $n = 10^6$ и больше выходит за его пределы, и обращения к памяти становятся медленнее. Это влияет на константу, но не на асимптотику: при дальнейшем росте n в 10 раз время растёт в 10,5 раза.

5 Выводы

Выполнив седьмую лабораторную работу по курсу «Дискретный анализ», я познакомился с методом динамического программирования и применил его к задаче о преобразовании числа в единицу с минимальной суммарной стоимостью операций.

Главное, что я понял: сначала нужно проверить, что задача обладает оптимальной подструктурой и перекрывающимися подзадачами, и только потом выбирать метод. Жадный выбор в этой задаче не работает, потому что локально выгодная операция (например, деление) не всегда ведёт к минимальной общей стоимости. Решение через рекуррентное соотношение $dp[i] = i + \min(dp[i-1], dp[i/2], dp[i/3])$ корректно, так как оптимальный путь для числа i состоит з первой операции и оптимального пути для полученного числа.

Тест производительности показал, почему запоминание подзадач необходимо. Наивная рекурсия пересчитывает одни и те же значения, и уже при $n = 750$ ей требуется около трёх секунд, тогда как динамическое программирование решает эту же задачу за единицы микросекунд. При $n = 10^7$ ДП работает около 0,05 с, а время на один элемент таблицы остаётся в пределах нескольких наносекунд, что согласуется с оценкой $O(n)$. Я также заметил, что на очень малых входах измерения неустойчивы из-за погрешности таймера и накладных расходов, а на больших заметно влияет кэш процессора, поэтому асимптотику нужно проверять на достаточно больших данных и с несколькими повторами замера.

Отдельно я научился восстанавливать не только значение оптимума, но и сам путь: для этого в каждой ячейке таблицы достаточно хранить операцию, на которой достигнут минимум, и пройти по ним от n до единицы. Для проверки корректности я сравнил результаты ДП с наивным перебором на малых n и пересчитал стоимость по восстановленной последовательности операций.

К практическим наблюдениям отнесу выбор между восходящим и нисходящим подходом. Рекурсия с запоминанием при $n = 10^7$ имела бы глубину порядка n и привела бы к переполнению стека, поэтому итеративное заполнение таблицы здесь надёжнее. Недостатком выбранного решения является память $O(n)$: при максимальном n таблица занимает около 80 МБ, и для более жёстких ограничений пришлось бы искать более экономный способ.

Метод динамического программирования применим во многих задачах: поиск наибольшей общей подпоследовательности, расстояние редактирования, задача о рюкзаке, кратчайшие пути в графах, разбиение последовательностей. Полученный опыт (определить состояние, записать рекуррентное соотношение, выбрать порядок вычисления, восстановить ответ) пригодится для решения таких задач.

Список литературы

[1] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ, 2-е издание.* — Издательский дом «Вильямс», 2007. Перевод с английского: И. В. Красиков, Н. А. Орехова, В. Н. Романов. — 1296 с. (ISBN 5-8459-0857-4 (рус.))

[2] *Динамическое программирование — Википедия.*
URL: `https://ru.wikipedia.org/wiki/Динамическое_программирование` (дата обращения: 25.10.2026).